

DMI COLLEGE OF ENGINEERING

(AN AUTONOMOUS INSTITUTION)

(APPROVED BY AICTE, AFFILIATED TO ANNA UNIVERSITY,
ACCREDITED BY NBA, ISO CERTIFIED INSTITUTION)

PALANCHUR – NAZARATHPET P.O., CHENNAI – 600 123

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



CS1253 DATABASE MANAGEMENT SYSTEMS

NAME OF STUDENT :

REGISTER NUMBER :

YEAR / CLASS : II / B.E CSE

SEMESTER : 04

ACADEMIC YEAR : 2026 (EVEN SEMESTER)

DMI COLLEGE OF ENGINEERING

(AN AUTONOMOUS INSTITUTION)

PALANCHUR – NAZARATHPET P.O., CHENNAI – 600 123

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



CERTIFICATE UNIVERSITY REGISTER NUMBER

Certified that this is Bonafide record of Practical work done by
Mr./Ms.....**COMPUTER SCIENCE AND
ENGINEERING** Department in the **CS1253 – DATABASE MANAGEMENT
SYSTEMS** Laboratory during the Semester **IV** academic year **2026** and submitted
for the University Practical Examination conducted onat
DMI COLLEGE OF ENGINEERING, PALANCHUR, CHENNAI– 600123.

SUBJECT IN-CHARGE

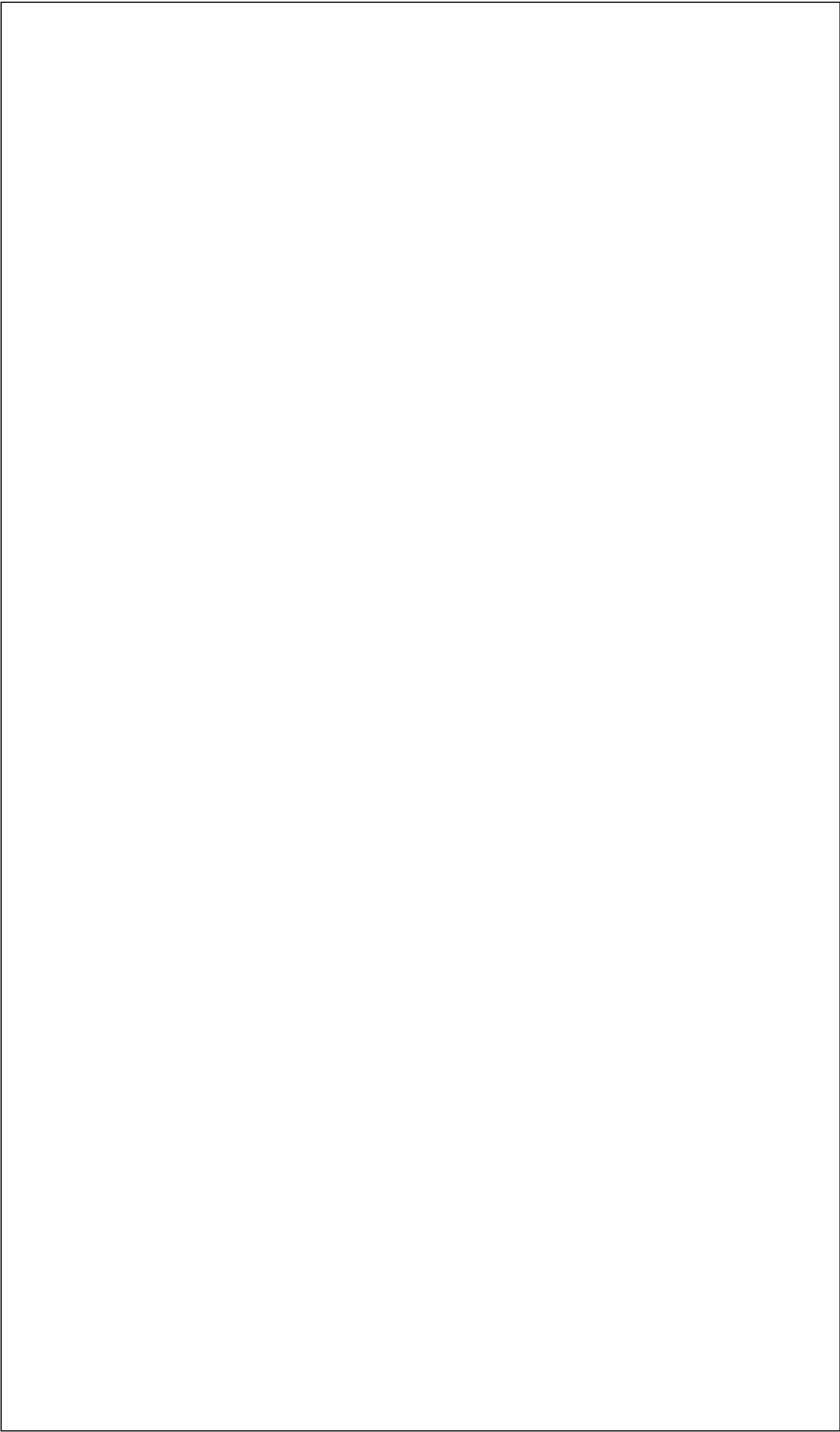
HEAD OF THE DEPARTMENT

INTERNAL EXAMINER

EXTERNAL EXAMINER

TABLE OF CONTENTS

S.No	DATE	NAME OF THE EXPERIMENT	PAGE NO	SIGNATURE
1				
2				
3				
4				
5				
6				
7				
8				
9				
10				



EX.NO: 1
DATE :

DDL AND DML COMMANDS

AIM

To execute basic command in MySQL using DDL and DML.

PROCEDURE

Step 1: Start

Step 2: Create a database and use it for basic operations.

Step 3: Create a table with necessary attributes and execute DDL and DML commands.

Step 4: Display the result.

Step 5: Stop

DDL (DATA DEFINITION LANGUAGE)

- CREATE
- ALTER
- DROP
- TRUNCATE
- COMMENT
- RENAME

SQL> CREATE TABLE EMP (EMPNO NUMBER (4), ENAME VARCHAR2 (10),
DESIGNATIN VARCHAR2 (10), SALARY NUMBER (8,2));

Table created.

SQL: DESC <TABLE NAME>;
SQL> DESC EMP;

Name	Null?	Type
-----		-----
EMPNO		NUMBER(4)
ENAME		VARCHAR2(10)
DESIGNATIN		VARCHAR2(10)
SALARY		NUMBER(8,2)

SQL>ALTER TABLE EMP MODIFY EMPNO NUMBER (6);

Table altered.

SQL> DESC EMP;

Name	Null?	Type
-----		-----
EMPNO		NUMBER(6)
ENAME		VARCHAR2(10)
DESIGNATIN		VARCHAR2(10)
SALARY		NUMBER(8,2)

SQL>ALTER TABLE EMP ADD (DOB DATE, DOJ DATE);
Table altered.
SQL> DESC
EMP;

Name	Null?	Type
EMPNO		NUMBER (7)
ENAME		VARCHAR 2(12)
DESIGNATIN		VARCHAR 2(10)
SALARY		NUMBER (8,2)
QUALIFICATION		VARCHAR 2(6)
DOB		DATE
DOJ		DATE

REMOVE / DROP

SQL> ALTER TABLE EMP DROP COLUMN DOJ;
SQL> DESC EMP;

Name	Null?	Type
EMPNO		NUMBER (7)
ENAME		VARCHAR 2(12)
DESIGNATIN		VARCHAR 2(10)
SALARY		NUMBER (8,2)
QUALIFICATION		VARCHAR 2(6)
DOB		DATE

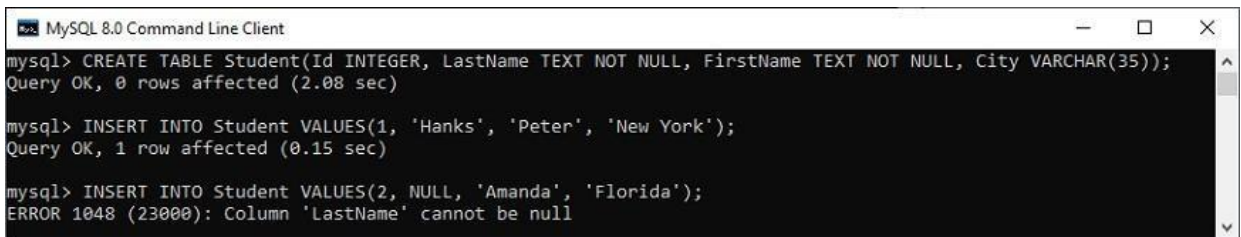
SQL>ALTER TABLE EMP DROP (DOB, QUALIFICATION);
Table altered.
SQL> DESC EMP;

Name	Null?	Type
EMPNO		NUMBER (7)
ENAME		VARCHAR 2(12)
DESIGNATIN		VARCHAR 2(10)
SALARY		NUMBER (8,2)

NOT NULL Constraint

MySQL> CREATE TABLE Student (Id INTEGER, Last Name TEXT NOT NULL, FirstName TEXT NOT NULL, City VARCHAR (35));
MySQL> INSERT INTO Student VALUES(1, 'Hanks', 'Peter', 'New York');
MySQL> INSERT INTO Student VALUES(2, NULL, 'Amanda', 'Florida');

Output



UNIQUE Constraint

```
MySQL> CREATE TABLE ShirtBrands(Id INTEGER, BrandName VARCHAR(40) UNIQUE, Size VARCHAR(30));
```

```
MySQL> INSERT INTO ShirtBrands(Id, BrandName, Size) VALUES(1, 'Pantaloons', 38), (2, 'Cantabil', 40);
```

```
MySQL> INSERT INTO ShirtBrands(Id, BrandName, Size) VALUES(1, 'Raymond', 38), (2, 'Cantabil', 40);
```

Output

```
MySQL 8.0 Command Line Client
mysql> CREATE TABLE ShirtBrands(Id INTEGER, BrandName VARCHAR(40) UNIQUE, Size VARCHAR(30));
Query OK, 0 rows affected (0.88 sec)

mysql> INSERT INTO ShirtBrands(Id, BrandName, Size) VALUES(1, 'Pantaloons', 38), (2, 'Cantabil', 40);
Query OK, 2 rows affected (0.26 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql> INSERT INTO ShirtBrands(Id, BrandName, Size) VALUES(3, 'Raymond', 38), (4, 'Cantabil', 40);
ERROR 1062 (23000): Duplicate entry 'Cantabil' for key 'shirtbrands.BrandName'
```

CHECK CONSTRAINT

CHECK (expr)

```
MySQL> CREATE TABLE Persons ( ID int NOT NULL, Name varchar(45) NOT NULL, Age int CHECK (Age>=18) );
```

```
MySQL> INSERT INTO Persons(Id, Name, Age)
VALUES (1,'Robert', 28), (2, 'Joseph', 35), (3, 'Peter', 40);
```

```
MySQL> INSERT INTO Persons(Id, Name, Age) VALUES (1,'Robert', 15);
```

Output

In the below output, we can see that the first INSERT query executes successfully, but the second statement fails and gives an error that says: CHECK constraint is violated for key Age.

```
MySQL 8.0 Command Line Client
mysql> CREATE TABLE Persons (
-> ID int NOT NULL,
-> Name varchar(45) NOT NULL,
-> Age int CHECK (Age>=18)
-> );
Query OK, 0 rows affected (0.87 sec)

mysql> INSERT INTO Persons(Id, Name, Age)
-> VALUES (1,'Robert', 28),
-> (2, 'Joseph', 35),
-> (3, 'Peter', 40);
Query OK, 3 rows affected (0.30 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> INSERT INTO Persons(Id, Name, Age) VALUES (1,'Robert', 15);
ERROR 3819 (HY000): Check constraint 'persons_chk_1' is violated.
```

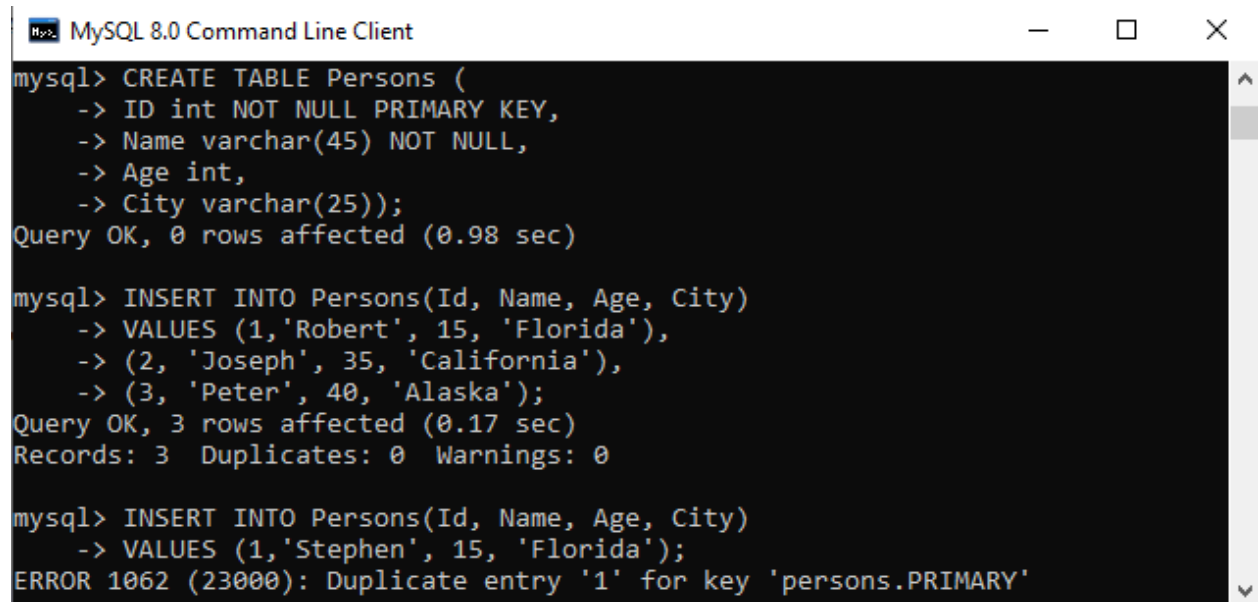
PRIMARY KEY CONSTRAINT

```
CREATE TABLE Persons ( ID int NOT NULL PRIMARY KEY, Name varchar(45) NOT NULL, Age int, City varchar(25));
```

```
INSERT INTO Persons(Id, Name, Age, City) VALUES (1,'Robert', 15, 'Florida') ,
(2, 'Joseph', 35, 'California'), (3, 'Peter', 40, 'Alaska');
```

```
INSERT INTO Persons(Id, Name, Age, City) VALUES (1,'Stephen', 15, 'Florida');
```

Output



```
MySQL 8.0 Command Line Client
mysql> CREATE TABLE Persons (
  -> ID int NOT NULL PRIMARY KEY,
  -> Name varchar(45) NOT NULL,
  -> Age int,
  -> City varchar(25));
Query OK, 0 rows affected (0.98 sec)

mysql> INSERT INTO Persons(Id, Name, Age, City)
  -> VALUES (1,'Robert', 15, 'Florida'),
  -> (2, 'Joseph', 35, 'California'),
  -> (3, 'Peter', 40, 'Alaska');
Query OK, 3 rows affected (0.17 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> INSERT INTO Persons(Id, Name, Age, City)
  -> VALUES (1,'Stephen', 15, 'Florida');
ERROR 1062 (23000): Duplicate entry '1' for key 'persons.PRIMARY'
```

RESULT

Thus, the queries were executed successfully.

EX.NO:2	FOREIGN KEY AND REFERENTIAL INTEGRITY CONSTRAINT
DATE :	

AIM

To create a set of tables and add foreign key and referential integrity constraints.

PROCEDURE

Step 1:Start

Step 2:Create Table Department and Employee with necessary attributes.

Step 3:Add Foreign Key constraints in department table by altering it.

Step 4:Check referential integrity constraints by perform any operation.

Step 5: Stop

DEPARTMENT

```
CREATE TABLE Department(  
Id INT PRIMARY KEY,  
Name NVARCHAR(50)  
);  
-- Insert some test data in Department Table  
Insert into Department values (10, 'IT');  
Insert into Department values (20, 'HR');  
Insert into Department values (30, 'INFRA');
```

EMPLOYEES

```
CREATE TABLE Employees(  
Id INT PRIMARY KEY,  
Name VARCHAR(100) NOT NULL,  
DepartmentID INT  
);  
  
-- Adding the Foreign Key Constraint  
ALTER TABLE Employees ADD FOREIGN KEY (DepartmentId) REFERENCES  
Department(Id);  
  
-- Insert some test data in Employees Table  
INSERT into Employees VALUES (101, 'Anurag', 10);  
INSERT into Employees VALUES (102, 'Pranaya', 20);  
INSERT into Employees VALUES (103, 'Hina', 30);
```

Delete from Parent Table

```
DELETE FROM Department WHERE Id = 10;
```

OUTPUT

SQL> DELETE from Department where Id=10;

ERROR 1451 (23000): Cannot delete or update a parent row: a foreign key constraint fails (`db2`.`employees`, CONSTRAINT `employees\_ibfk\_1` FOREIGN KEY (`DepartmentID`) REFERENCES `department` (`Id`))

RESULT

Thus the queries were executed successfully

EX.NO: 3

QUERIES WITH WHERE CLAUSE AND AGGREATE FUNCTIONS.

DATE :

AIM

To write queries using WHERE clause and Aggreate Functions.

PROCEDURE

Step 1: Start

Step 2: Write queries using different WHERE Clause

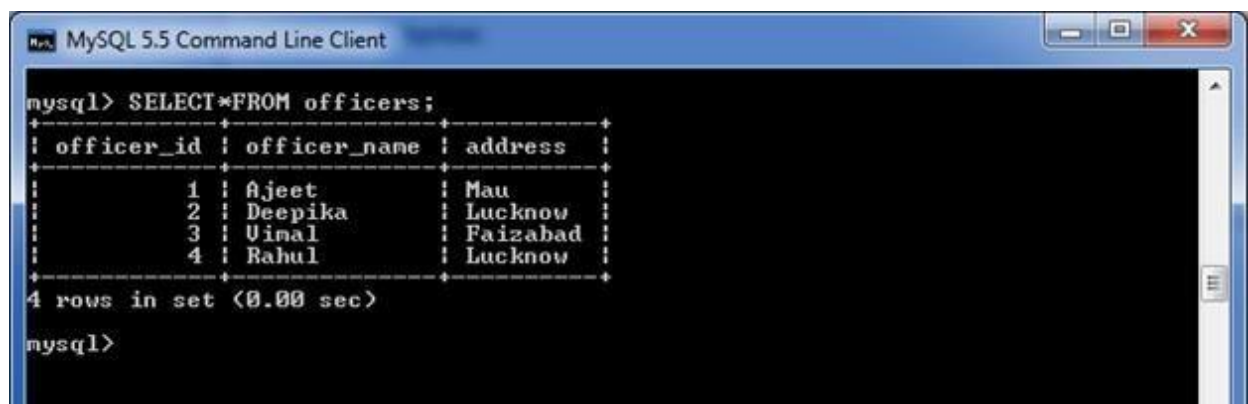
Step 3: Write queries for Aggregate functions like count, avg, min, max

Step 4: stop

MySQL WHERE Clause

Syntax:

Select * from Tablename WHERE conditions;



```
mysql> SELECT * FROM officers;
```

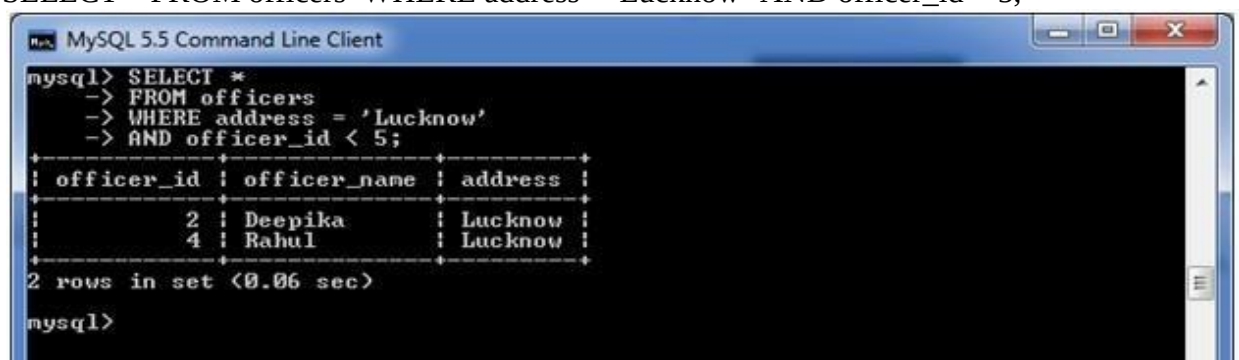
officer_id	officer_name	address
1	Ajeet	Mau
2	Deepika	Lucknow
3	Uinal	Faizabad
4	Rahul	Lucknow

```
4 rows in set (0.00 sec)

mysql>
```

MySQL WHERE Clause with AND condition

SELECT * FROM officers WHERE address = 'Lucknow' AND officer_id < 5;



```
mysql> SELECT *
-> FROM officers
-> WHERE address = 'Lucknow'
-> AND officer_id < 5;
```

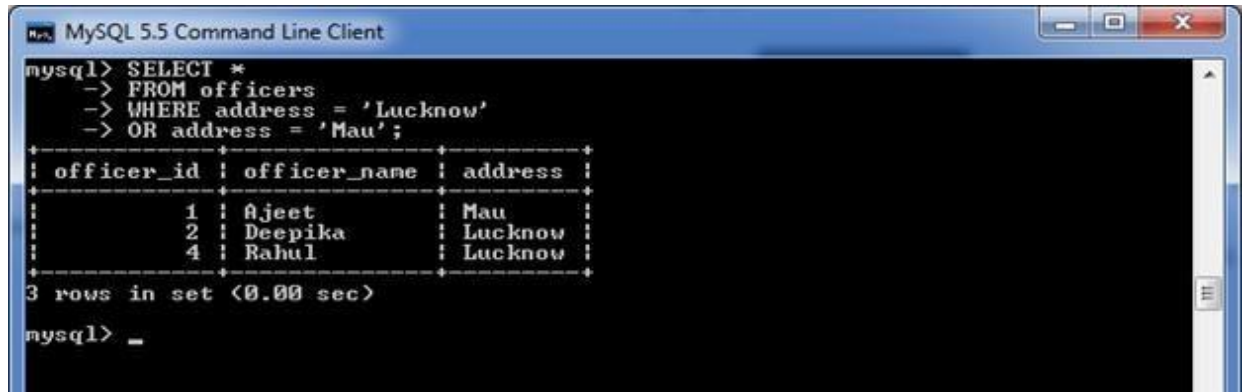
officer_id	officer_name	address
2	Deepika	Lucknow
4	Rahul	Lucknow

```
2 rows in set (0.06 sec)

mysql>
```

WHERE Clause with OR condition

SELECT * FROM officers WHERE address = 'Lucknow' OR address = 'Mau';



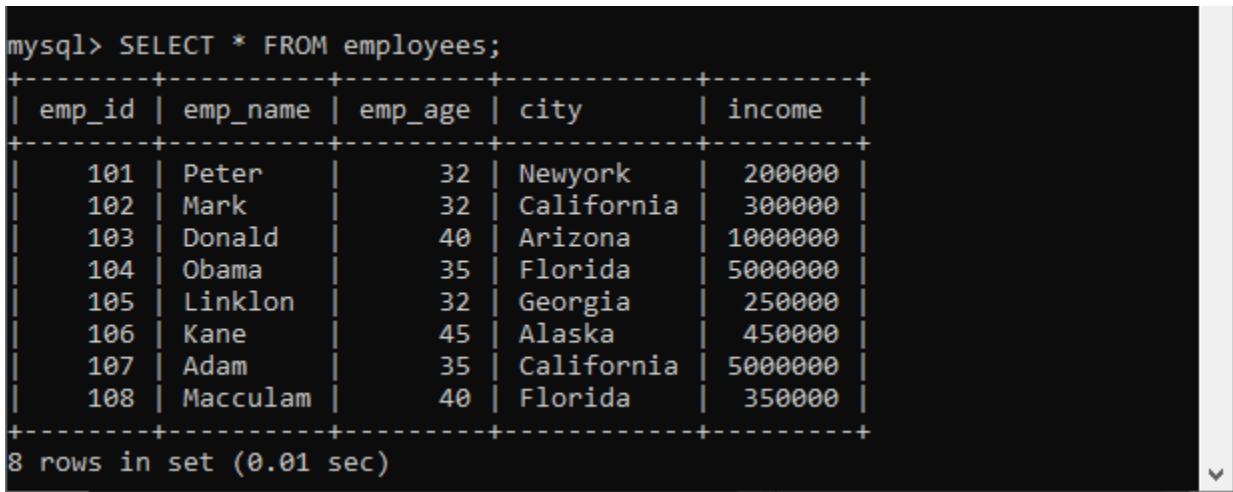
MySQL WHERE Clause with combination of AND & OR conditions

SELECT * FROM officers WHERE (address = 'Mau' AND officer_name = 'Ajeet')
OR (officer_id < 5);



AGGREGATE FUNCTIONS

Consider a table named "employees" that contains the following data.



MySQL> SELECT COUNT(emp_name) FROM employees;

Output:

```
MySQL 8.0 Command Line Client

mysql> SELECT COUNT(emp_name) FROM employees;
+-----+
| COUNT(emp_name) |
+-----+
|                8 |
+-----+
1 row in set (0.00 sec)
```

MySQL> **SELECT** COUNT(*) **FROM** employees **WHERE** emp_age>32;

Output:

```
MySQL 8.0 Command Line Client

mysql> SELECT COUNT(*) FROM employees WHERE emp_age>32;
+-----+
| COUNT(*) |
+-----+
|         5 |
+-----+
1 row in set (0.00 sec)
```

Consider our database has a table named **employees**, having the following data. Now, we are going to understand this function with various examples:

```
MySQL 8.0 Command Line Client

mysql> SELECT * FROM employees;
+-----+-----+-----+-----+-----+
| emp_id | emp_name | occupation | working_date | working_hours |
+-----+-----+-----+-----+-----+
| 1      | Joseph  | Business  | 2020-04-10   | 10             |
| 2      | Stephen | Doctor    | 2020-04-10   | 15             |
| 3      | Mark    | Engineer  | 2020-04-10   | 12             |
| 4      | Peter   | Teacher   | 2020-04-10   | 9              |
| 1      | Joseph  | Business  | 2020-04-12   | 10             |
| 2      | Stephen | Doctor    | 2020-04-12   | 15             |
| 4      | Peter   | Teacher   | 2020-04-12   | 9              |
| 3      | Mark    | Engineer  | 2020-04-12   | 12             |
| 1      | Joseph  | Business  | 2020-04-14   | 10             |
| 4      | Peter   | Teacher   | 2020-04-14   | 9              |
+-----+-----+-----+-----+-----+
10 rows in set (0.00 sec)
```

MySQL> **SELECT** SUM(working_hours) **AS** "Total working hours" **FROM** employees;

Output:

```
MySQL 8.0 Command Line Client
mysql> SELECT SUM(working_hours) AS "Total working hours" FROM employees;
+-----+
| Total working hours |
+-----+
|          111 |
+-----+
1 row in set (0.00 sec)
```

MySQL avg() function example

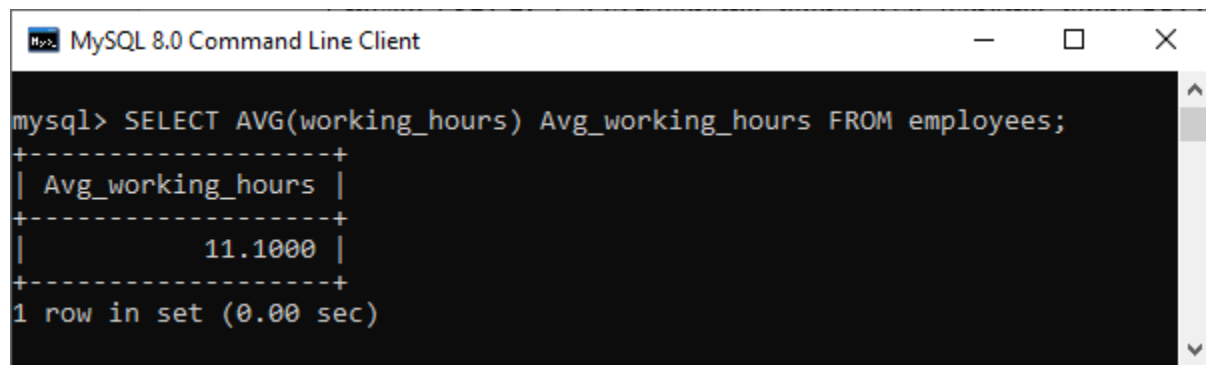
Consider our database has a table named **employees**, having the following data. Now, we are going to understand this function with various examples:

```
MySQL 8.0 Command Line Client
mysql> SELECT * FROM employees;
+-----+-----+-----+-----+-----+
| emp_id | emp_name | occupation | working_date | working_hours |
+-----+-----+-----+-----+-----+
| 1 | Joseph | Business | 2020-04-10 | 10 |
| 2 | Stephen | Doctor | 2020-04-10 | 15 |
| 3 | Mark | Engineer | 2020-04-10 | 12 |
| 4 | Peter | Teacher | 2020-04-10 | 9 |
| 1 | Joseph | Business | 2020-04-12 | 10 |
| 2 | Stephen | Doctor | 2020-04-12 | 15 |
| 4 | Peter | Teacher | 2020-04-12 | 9 |
| 3 | Mark | Engineer | 2020-04-12 | 12 |
| 1 | Joseph | Business | 2020-04-14 | 10 |
| 4 | Peter | Teacher | 2020-04-14 | 9 |
+-----+-----+-----+-----+-----+
10 rows in set (0.00 sec)
```

MySQL> **SELECT** AVG(working_hours) Avg_working_hours **FROM** employees;

Output:

We will get the result as below:



```
mysql> SELECT AVG(working_hours) Avg_working_hours FROM employees;
+-----+
| Avg_working_hours |
+-----+
|          11.1000 |
+-----+
1 row in set (0.00 sec)
```

RESULT

Thus the queries were executed successfully

EX.NO:4
DATE :

SIMPLE JOIN AND SUB QUERIES

AIM:

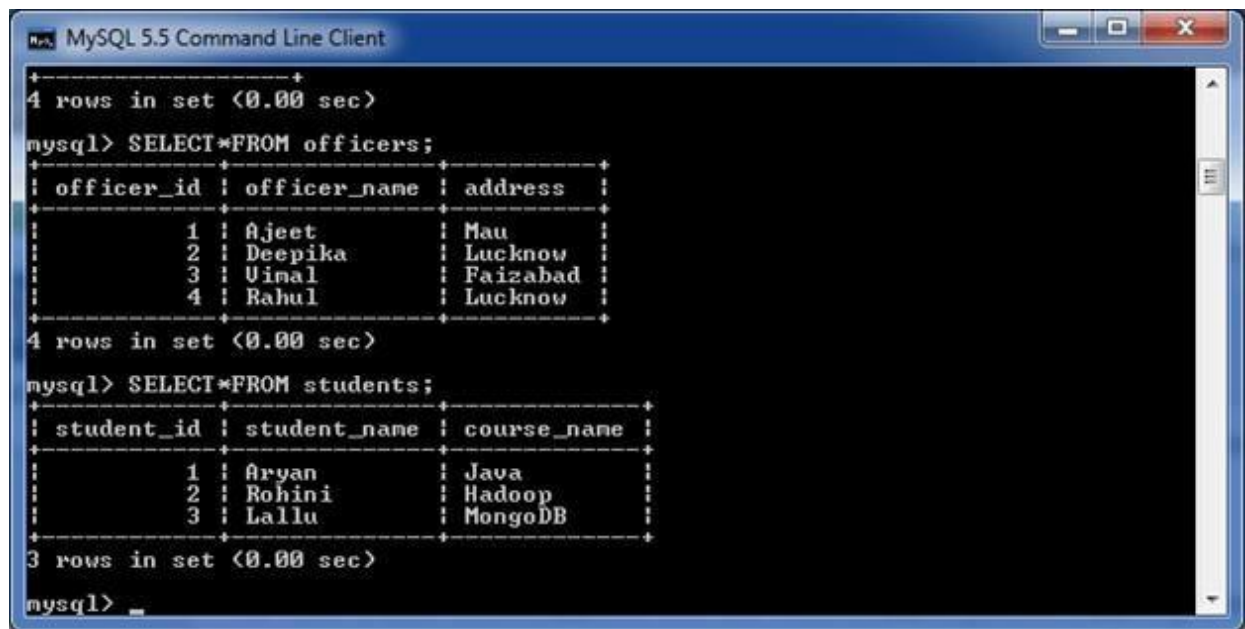
To execute and verify the SQL commands for Simple JOIN and sub queries.

PROCEDURE

- STEP 1: Start
- STEP 2: Create the table with its essential attributes.
- STEP 3: Insert attribute values into the table
- STEP 4: Execute Commands for JOIN operation and extract information from the table.
- STEP 5: Execute Commands for Sub queries operation.
- STEP 6: Stop

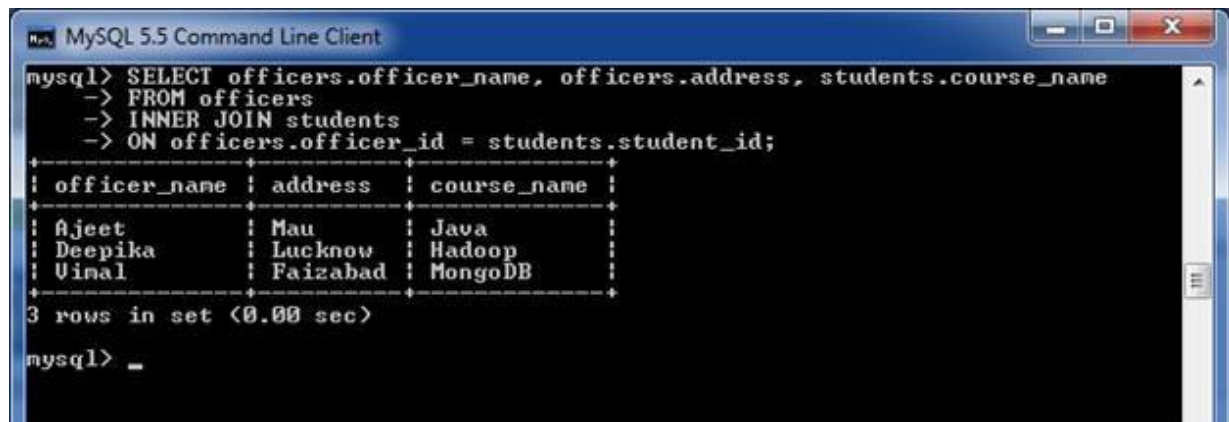
MYSQL INNER JOIN (SIMPLE JOIN)

Consider two tables "officers" and "students", having the following data.



SQL> SELECT officers.officer_name, officers.address, students.course_name
FROM officers INNER JOIN students ON officers.officer_id = students.student_id;

Output



MYSQL SUBOUERY

MySQL 8.0 Command Line Client

mysql> SELECT * FROM employees;

emp_id	emp_name	emp_age	city	income
101	Peter	32	Newyork	200000
102	Mark	32	California	300000
103	Donald	40	Arizona	1000000
104	Obama	35	Florida	5000000
105	Linklon	32	Georgia	250000
106	Kane	45	Alaska	450000
107	Adam	35	California	5000000
108	Macculam	40	Florida	350000
109	Brayan	32	Alaska	400000
110	Stephen	40	Arizona	600000
111	Alexander	45	California	70000

SQL>SELECT emp_name, city, income FROM employees WHERE emp_id IN (SELECT e
mp_id FROM employees);

MySQL 8.0 Command Line Client

mysql> SELECT emp_name, city, income FROM employees
-> WHERE emp_id IN (SELECT emp_id FROM employees);

emp_name	city	income
Peter	Newyork	200000
Mark	California	300000
Donald	Arizona	1000000
Obama	Florida	5000000
Linklon	Georgia	250000
Kane	Alaska	450000
Adam	California	5000000
Macculam	Florida	350000
Brayan	Alaska	400000
Stephen	Arizona	600000
Alexander	California	70000

RESULT

Thus the queries were executed successfully

EX.NO :5
DATE :

NATURAL JOIN,EQUI JOIN AND OUTER JOIN

AIM

To write a query to perform natural join ,equi join and outer join.

PROCEDURE

Step 1: Start

Step 2:Create table with necessary attributes .

Step 3: Perform natural join,equi join and outer join operations with queries

Step 4: Stop

Syntax:

SELECT [column_names | *] FROM table_name1 NATURAL JOIN table_name2;

/ -- Table name: customer -*/*

CREATE TABLE customer (id **INT** **AUTO_INCREMENT PRIMARY KEY**,
customer_name **VARCHAR**(55), account **int**, email **VARCHAR**(55));

/ -- Table name: balance -*/*

CREATE TABLE balance (id **INT** **AUTO_INCREMENT PRIMARY KEY**,
account **int**, balance **FLOAT**(10, 2));

/ -- Data for customer table -*/*

INSERT INTO customer(customer_name, account, email) **VALUES**('Stephen', 1030, 'stephen@javatpoint.com'), ('Jenifer', 2035, 'jenifer@javatpoint.com'), ('Mathew', 5564, 'mathew@javatpoint.com'), ('Smith', 4534, 'smith@javatpoint.com'), ('David', 7648, 'david@javatpoint.com');

/ -- Data for balance table -*/*

INSERT INTO balance(account, balance)
VALUES(1030, 50000.00), (2035, 230000.00), (5564, 125000.00), (4534, 80000.00),
(7648, 45000.00);

NATURAL JOIN:

MySQL> **SELECT** cust. customer_name, bal.balance **FROM** customer **AS** cust **NATURAL JOIN** balance **AS** bal;

```
MySQL 8.0 Command Line Client

mysql> SELECT cust. customer_name, bal.balance
-> FROM customer AS cust
-> NATURAL JOIN balance AS bal;
+-----+-----+
| customer_name | balance |
+-----+-----+
| Stephen       | 50000.00 |
| Jenifer       | 230000.00 |
| Mathew        | 125000.00 |
| Smith         | 80000.00 |
| david         | 45000.00 |
+-----+-----+
5 rows in set (0.00 sec)
```

MYSQL RIGHT OUTER JOIN

Syntax:

SELECT columns FROM table1 RIGHT [OUTER] JOIN table2 ON table1.column = table2.column;

Consider two tables "officers" and "students", having the following data.

```
MySQL 5.5 Command Line Client

4 rows in set (0.00 sec)

mysql> SELECT*FROM officers;
+-----+-----+-----+
| officer_id | officer_name | address |
+-----+-----+-----+
| 1 | Ajeet | Mau |
| 2 | Deepika | Lucknow |
| 3 | Uimal | Faizabad |
| 4 | Rahul | Lucknow |
+-----+-----+-----+
4 rows in set (0.00 sec)

mysql> SELECT*FROM students;
+-----+-----+-----+
| student_id | student_name | course_name |
+-----+-----+-----+
| 1 | Aryan | Java |
| 2 | Rohini | Hadoop |
| 3 | Lallu | MongoDB |
+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> _
```

MySQL>SELECT officers.officer_name, officers.address, students.course_name, students.student_name FROM officers RIGHT JOIN students ON officers.officer_id = students.student_id;

Output

```
MySQL 5.5 Command Line Client

mysql>
mysql> SELECT officers.officer_name, officers.address, students.course_name, st
students.student_name
-> FROM officers
-> RIGHT JOIN students
-> ON officers.officer_id = students.student_id;
+-----+-----+-----+-----+
| officer_name | address | course_name | student_name |
+-----+-----+-----+-----+
| Ajeet       | Mau    | Java    | Aryan    |
| Deepika     | Lucknow | Hadoop  | Rohini   |
| Uimal       | Faizabad | MongoDB | Lallu    |
+-----+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> _
```

EQUI JOIN

SELECT column_name (s) FROM table_name1, table_name2,..... , table_nameN

WHERE table_name1.column_name = table_name2.column_name;

Consider two tables named **customer** and **balance**

MySQL 8.0 Command Line Client

```
mysql> select * from customer;
+-----+-----+-----+-----+
| id | customer_name | account | email |
+-----+-----+-----+-----+
| 1 | Stephen      | 1030    | stephen@javatpoint.com |
| 2 | Jenifer      | 2035    | jenifer@javatpoint.com |
| 3 | Mathew       | 5564    | mathew@javatpoint.com  |
| 4 | Smith        | 4534    | smith@javatpoint.com   |
| 5 | david        | 7648    | david@javatpoint.com   |
+-----+-----+-----+-----+
5 rows in set (0.00 sec)

mysql> select * from balance;
+-----+-----+-----+
| id | account_num | balance |
+-----+-----+-----+
| 1 | 1030        | 50000.00 |
| 2 | 2035        | 230000.00 |
| 3 | 5564        | 125000.00 |
| 4 | 4534        | 80000.00 |
| 5 | 7648        | 45000.00 |
+-----+-----+-----+
5 rows in set (0.00 sec)
```

MySQL> **SELECT** cust. customer_name, bal.balance **FROM** customer **AS** cust, balance **AS** bal **WHERE** cust.account = bal.account_num;

MySQL 8.0 Command Line Client

```
mysql> SELECT cust. customer_name, bal.balance
-> FROM customer AS cust, balance AS bal
-> WHERE cust.account = bal.account_num;
+-----+-----+
| customer_name | balance |
+-----+-----+
| Stephen      | 50000.00 |
| Jenifer      | 230000.00 |
| Mathew       | 125000.00 |
| Smith        | 80000.00 |
| david        | 45000.00 |
+-----+-----+
5 rows in set (0.00 sec)
```

RESULT

Thus the queries were executed successfully

EX.NO:6

PROCEDURE AND FUNCTIONS

DATE :

PROCEDURE

AIM :

To write a SQL block to display the student name, marks whose average mark is above 60%.

ALGORITHM

STEP 1:Start

STEP 2:Create a table with table name stud_exam

STEP 3:Insert the values into the table and Calculate total and average of each student

STEP 4: Execute the procedure function the student who get above 60%.

STEP 5: Display the total and average of student

STEP 6: End

SETTING SERVEROUTPUT ON:

SQL> SET SERVEROUTPUT ON

PROGRAM:

PROCEDURE USING POSITIONAL PARAMETERS:

```
SQL> SET SERVEROUTPUT ON
SQL> CREATE OR REPLACE PROCEDURE PROC1 AS
2 BEGIN
3   DBMS_OUTPUT.PUT_LINE('Hello from procedure...');
4 END;
5 /
```

Output

Procedure created.

```
SQL> EXECUTE PROC1
Hello from procedure...
```

PL/SQL procedure successfully completed.

```
SQL> create table student(regno number(4),name varchar2(20),mark1 number(3), mark2
number(3), mark3 number(3), mark4 number(3), mark5 number(3));
```

Table created

```
SQL> insert into student values (101,'priya', 78, 88,77,60,89);
```

1 row created.

SQL> insert into student values (102,'surya', 99,77,69,81,99); 1 row created

SQL> insert into student values (103,'suryapriya', 100,90,97,89,91);

1 row created.

SQL> select * from student;

regno	name	mark1	mark2	mark3	mark4	mark5
101	priya	78	88	77	60	89
102	surya	99	77	69	81	99
103	suryapriya	100	90	97	89	91

```
SQL> declare
2  ave number(5,2);
3  tot number(3);
4  cursor c_mark is select*from student where mark1>=40 and mark2>=40 and
5  mark3>=40 and mark4>=40 and mark5>=40;
6  begin
7  dbms_output.put_line('regno name mark1 mark2 mark3 mark4 mark4 mark5 total
8  average');
9  dbms_output.put_line('.....');
10 for student in c_mark
11 loop
12 tot:=student.mark1+student.mark2+student.mark3+student.mark4+student.mark5;
13 ave:=tot/5;
14 dbms_output.put_line(student.regno||rpad(student.name,15)
15 ||rpad(student.mark1,6)||rpad(student.mark2,6)||rpad(student.mark3,6)
16 ||rpad(student.mark4,6)||rpad(student.mark5,6)||rpad(tot,8)||rpad(ave,5));
17 end loop;
18 end;
19 /
```

OUTPUT

regno	name	mark1	mark2	mark3	mark4	mark5	total	average
101	priya	78	88	77	60	89	393	79
102	surya	99	77	69	81	99	425	85
103	suryapriya	100	90	97	89	91	467	93

PL/SQL procedure successfully completed.

FUNCTIONS

AIM

To write a Functional procedure to search an address from the given database.

PROCEDURE

STEP 1: Start

STEP 2: Create the table with essential attributes.

STEP 3: Initialize the Function to carryout the searching procedure..

STEP 4: Frame the searching procedure for both positive and negative searching.

STEP 5: Execute the Function for both positive and negative result .

STEP 6: Stop

SQL> create table phonebook (phone_no number (6) primary key,username
varchar2(30),doorno varchar2(10),
street varchar2(30),place varchar2(30),pincode char(6));

Table created.

SQL> insert into phonebook values(20312,'vijay','120/5D','bharathi street','NGO
colony','629002');

1 row created.

SQL> insert into phonebook values(29467,'vasanth','39D4','RK bhavan','sarakkal
vilai','629002');

1 row created.

SQL> select * from phonebook;

PHONE_NO	USERNAME	DOORNO	STREET	PLACE	PINCODE
20312	vijay	120/5D	bharathi street	NGO colony	629002
29467	vasanth	39D4	RK bhavan	sarakkal vilai	629002

SQL> create or replace function findAddress(phone in number) return varchar2 as
address varchar2(100);

begin
select username||','||doorno ||','||street ||','||place||','||pincode into address from phonebook
where phone_no=phone;
return address;
exception
when no_data_found then return 'address not found';
end;
/

Function created.

SQL>declare

```
2 address varchar2(100);
3 begin
4 address:=findaddress(20312);
5 dbms_output.put_line(address);
6 end;
7 /
```

OUTPUT

Vijay,120/5D,bharathi street,NGO colony,629002

RESULT

Thus the PL/SQL procedure successfully completed.

AIM

To write a query to perform DCL and TCL commands.

PROCEDURE

- Step 1: Start
- Step 2: Create table with necessary attributes.
- Step 3: Perform DCL query like GRANT and REVOKE
- Step 4: Perform TCL like SAVEPOINT,ROLLBACK and COMMIT.
- Step 5: Stop.

DCL COMMANDS

GRANT

GRANT privilege_name ON object_name TO {user_name |PUBLIC |role_name}
[WITH GRANT OPTION];

MySQL> GRANT SELECT ON employee TO
user1;Command Successfully Completed

REVOKE

REVOKE privilege_name ON object_name FROM {user_name |PUBLIC |role_name}

MySQL> REVOKE SELECT ON employee FROM
user1;Command Successfully Completed

TCL(TRNSACTION CONTROL LANGUAGE)

SQL> SAVEPOINT S1;
Savepoint created.

SQL> SELECT * FROM EMP;

EMPNO	ENAME	DESIGNATIN	SALARY
-----	-----	-----	-----
101	NAGARAJAN	LECTURER	16000
102	SARAVANAN	ASST. PROF	16000
104	CHINNI	HOD, PROF	45000

SQL> INSERT INTO EMP VALUES(105,'PARTHASAR','STUDENT',100);

1 row created.

SQL> SELECT * FROM EMP;

EMPNO	ENAME	DESIGNATIN	SALARY
-----	-----	-----	-----
105	PARTHASAR	STUDENT	100
101	NAGARAJAN	LECTURER	16000
102	SARAVANAN	ASST. PROF	16000
104	CHINNI	HOD, PROF	45000

ROLL BACK

SQL> ROLL BACK S1;

Rollback complete.

SQL> SELECT * FROM EMP;

EMPNO	ENAME	DESIGNATIN	SALARY
-----	-----	-----	-----
101	NAGARAJAN	LECTURER	16000
102	SARAVANAN	ASST. PROF	16000
104	CHINNI	HOD, PROF	45000

COMMIT

SQL> COMMIT;

Commit complete.

RESULT

Thus the queries were executed successfully

EX.NO:8	CREATION OF DATABASE TRIGGERS
DATE :	

AIM

To create database triggers using PL/SQL code

PROCEDURE

- STEP 1: Creates a trigger for insertion of each row.
- STEP 2: Declare a cursor which contains the roll number field
- STEP 3: Before insertion check of the roll number already exists in the table
- STEP 4: If it exists raise an application error and display “roll no exists”.
- STEP 5: Else perform insertion

SYNTAX

create or replace trigger trigger name [before/after] {DML
statements} on [table name] [for each row/statement] begin

exception
end;

PROGRAM

SQL>create table poo(rno number(5),name varchar2(10));
Table created.
SQL>insert into poo values (01.‘kala’);
1 row created.
SQL>select * from poo;

RNO	NAME
-----	-----
1	kala
2	priya

SQL>create or replace trigger pool before insert on poo for each row
2 declare
3 rno poo.rno%type
4 cursor c is select rno from poo;
5 begin

```
6 open c;
7 loop;
8 fetch c into rno;
9 if:new.rno=rno then
10 raise_application_error(-20005,"rno already exist");
11 end if;
12 exit when c%NOTFOUND
13 end loop;
14 close c;
15 end;
16 /
Trigger created.
```

OUTPUT

```
SQL>insert into poo values(01,"kala")
Insert into poo values (01,"kala")
*
ERROR at line1:
ORA-20005:rno already exist
ORA-06512:"SECONDCSEA.POOL",line 9
ORA-04088:error during execution at trigger "SECONDCSEA.POOL"
```

RESULT

Thus the PL/SQL blocks are developed for triggers and the results are verified.

EX.NO:9
DATE

VIEWS AND INDEX

AIM

To execute and verify the SQL commands for Views and Indexes.

PROCEDURE

STEP 1: Start

STEP 2: Create the table with its essential attributes.

STEP 3: Insert attribute values into the table.

STEP 4: Create the view from the above created table.

STEP 5: Execute different Commands and extract information from the View.

STEP 6: Stop

CREATION OF TABLE

```
SQL> CREATE TABLE EMPLOYEE (  
      EMPLOYEE_NAME VARCHAR2(10),  
      EMPLOYEE_NO NUMBER(8),  
      DEPT_NAME VARCHAR2(10),  
      DEPT_NO NUMBER (5),DATE_OF_JOIN DATE);
```

Table created.

TABLE DESCRIPTION

```
-----  
SQL> DESC EMPLOYEE;
```

NAME	NULL?	TYPE
-----	-----	-----
EMPLOYEE_NAME		VARCHAR2(10)
EMPLOYEE_NO		NUMBER(8)
DEPT_NAME		VARCHAR2(10)
DEPT_NO		NUMBER(5)
DATE_OF_JOIN		DATE

CREATION OF VIEW

```
-----  
SQL> CREATE VIEW EMPVIEW AS SELECT  
EMPLOYEE_NAME,EMPLOYEE_NO,DEPT_NAME,DEPT_NO,DATE_OF_JOIN FROM  
EMPLOYEE;
```

view created.

DESCRIPTION OF VIEW

```
-----  
SQL> DESC EMPVIEW;
```

NAME	NULL?	TYPE
-----	-----	-----
EMPLOYEE_NAME		VARCHAR2(10)
EMPLOYEE_NO		NUMBER(8)
DEPT_NAME		VARCHAR2(10)
DEPT_NO		NUMBER(5)

DISPLAY VIEW

SQL> SELECT * FROM EMPVIEW;

EMPLOYEE_N	EMPLOYEE_NO	DEPT_NAME	DEPT_NO

RAVI	124	ECE	89
VIJAY	345	CSE	21
RAJ	98	IT	22
GIRI	100	CSE	67

INSERTION INTO VIEW

SQL> INSERT INTO EMPVIEW VALUES ('SRI', 120,'CSE', 67,'16-NOV-1981');

1 ROW CREATED.

SQL> SELECT * FROM EMPVIEW;

EMPLOYEE_N	EMPLOYEE_NO	DEPT_NAME	DEPT_NO

RAVI	124	ECE	89
VIJAY	345	CSE	21
RAJ	98	IT	22
GIRI	100	CSE	67
SRI	120	CSE	67

SQL> SELECT * FROM EMPLOYEE;

EMPLOYEE_N	EMPLOYEE_NO	DEPT_NAME	DEPT_NO	DATE_OF_J
-----				-----
RAVI	124	ECE	89	15-JUN-05
VIJAY	345	CSE	21	21-JUN-06
RAJ	98	IT	22	30-SEP-06
GIRI	100	CSE	67	14-NOV-81
SRI	120	CSE	67	16-NOV-81

DELETION OF VIEW

DELETE STATEMENT

SQL> DELETE FROM EMPVIEW WHERE EMPLOYEE_NAME='SRI';

SQL> SELECT * FROM EMPVIEW;

EMPLOYEE_N	EMPLOYEE_NO	DEPT_NAME	DEPT_NO

RAVI	124	ECE	89
VIJAY	345	CSE	21
RAJ	98	IT	22
GIRI	100	CSE	67

UPDATE STATEMENT:

SQL> UPDATE EMPKAVIVIEW SET EMPLOYEE_NAME='KAVI' WHERE EMPLOYEE_NAME='RAVI';

1 ROW UPDATED.

SQL> SELECT * FROM EMPKAVIVIEW;

EMPLOYEE_N	EMPLOYEE_NO	DEPT_NAME	DEPT_NO

KAVI	124	ECE	89
VIJAY	345	CSE	21
RAJ	98	IT	22
GIRI	100	CSE	67

DROP A VIEW:

SQL>DROP VIEW EMPVIEW;

VIEW DROPED

CREATE INDEX

MySQL> CREATE DATABASE
indexes;Query OK, 1 row affected (0.01
sec)

USE indexes;

Database changed

MySQL>CREATE TABLE

employees (employee_id int,

first_name varchar(50),

last_name varchar(50),

device_serial varchar(15), salary int);

Query OK, 0 rows affected (0.00 sec)

INSERT INTO employees VALUES

(1, 'John', 'Smith', 'ABC123', 60000), (2, 'Jane', 'Doe', 'DEF456', 65000),

(3, 'Bob', 'Johnson', 'GHI789', 70000), (4, 'Sally', 'Fields', 'JKL012', 75000),

(5, 'Michael', 'Smith', 'MNO345', 80000), (6, 'Emily', 'Jones', 'PQR678', 85000),

(7, 'David', 'Williams', 'STU901', 90000), (8, 'Sarah', 'Johnson', 'VWX234', 95000),

(9, 'James', 'Brown', 'YZA567', 100000);

Query OK, 9 rows affected (0.010 sec)

Records: 9 Duplicates: 0 Warnings: 0

MySQL>CREATE INDEX salary ON employees(salary);

Mqsql>EXPLAIN SELECT * FROM employees WHERE salary = 100000;

+		+		+		+		+		+		+		+		+		+		+		+	
id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered													
1	SIMPLE	employees	NULL	ref	salary	salary	5	const	1	100.00													
+		+		+		+		+		+		+		+		+		+		+		+	

1 row in set, 1 warning (0.00 sec)

RESULT

Thus views and indexes created successfull

EX.NO:10

XML DATABASE CREATION AND VALIDATION

DATE :

Aim

To create a XML database file and Validate the Schema

Algorithm

Step 1: Start

Step 2:Open MySQL command

prompt(version.5.5) Step 3:Create new database as

bookstore and use it.

Step 4:Create XML Schema for data values and load values

Step 5:Validate XML using ExtractValue function.

Step 6:Stop

CREATE TABLE

```
CREATE TABLE person (  
  
person_id INT NOT NULL PRIMARY KEY,  
  
fname VARCHAR(40) NULL,  
  
lname VARCHAR(40) NULL,  
  
created TIMESTAMP  
  
);
```

XML FILE PERSON.XML

```
<list>  
<personperson_id="1"fname="Kapek"lname="Sainnouine"/>  
<personperson_id="2"fname="Sajon"lname="Rondela"/>  
<personperson_id="3"><fname>Likame</fname><lname>Örrtmons</lname></person>  
<personperson_id="4"><fname>Slar</fname><lname>Manlanth</lname></person>  
<person><fieldname="person_id">5</field><fieldname="fname">Stoma</field>  
<fieldname="lname">Milu</field></person>  
<person><fieldname="person_id">6</field><fieldname="fname">Nirtam</field>  
<fieldname="lname">Sklöd</field></person>  
<personperson_id="7"><fname>Sungam</fname><lname>Dulbåd</lname></person>  
<personperson_id="8"fname="Sraref"lname="Encmelt"/>  
</list>
```

INSERT VALUES USING LOADXMLDATAFILE

LOAD XML LOCAL INFILE 'c:/db/person.xml' //this is ths location of the xml data file

INTO TABLE person

ROWS IDENTIFIED BY '<person>';

OUTPUT

MySQL>Select * from person;

```
mysql> LOAD XML LOCAL INFILE 'c:/db/person.xml'
-> INTO TABLE person
-> ROWS IDENTIFIED BY '<person>';
Query OK, 8 rows affected (0.03 sec)
Records: 8 Deleted: 0 Skipped: 0 Warnings: 0

mysql> select * from person;
+-----+-----+-----+-----+
| person_id | fname | lname | created |
+-----+-----+-----+-----+
| 1 | Kapek | Sainnouine | 2023-02-23 01:17:05 |
| 2 | Sajon | Rondela | 2023-02-23 01:17:05 |
| 3 | Likame | Örrtmons | 2023-02-23 01:17:05 |
| 4 | Slar | Manlanth | 2023-02-23 01:17:05 |
| 5 | Stoma | Milu | 2023-02-23 01:17:05 |
| 6 | Nirtam | Sklöð | 2023-02-23 01:17:05 |
| 7 | Sungam | Dulbåd | 2023-02-23 01:17:05 |
| 8 | Sraref | Encmelt | 2023-02-23 01:17:05 |
+-----+-----+-----+-----+
8 rows in set (0.00 sec)
```

VALIDATE XML USING EXTRACTVALUE FUNCTION

MySQL> SELECT

ExtractValue('<?xml version="1.0" encoding="UTF-8"?>

```
mysql> use bookstore;
Database changed
mysql> SELECT
-> ExtractValue('<?xml version="1.0" encoding="UTF-8"?>
->
-> <person person_id="1" fname="Kapek" lname="Sainnouine"/>
-> <person person_id="2" fname="Sajon" lname="Rondela"/>
-> <person person_id="3"><fname>Likame</fname><lname>Örrtmons</lname></person>
-> <person person_id="4"><fname>Slar</fname><lname>Manlanth</lname></person>
-> <person><field name="person_id">5</field><field name="fname">Stoma</field>
-> <field name="lname">Milu</field></person>
-> <person><field name="person_id">6</field><field name="fname">Nirtam</field>
-> <field name="lname">Sklöð</field></person>
-> <person person_id="7"><fname>Sungam</fname><lname>Dulbåd</lname></person>
-> <person person_id="8" fname="Sraref" lname="Encmelt"/>', '//fname//person_id');
+-----+-----+-----+-----+
| ExtractValue('<?xml version="1.0" encoding="UTF-8"?>
|
| <person person_id="1" fname="Kapek" lname="Sainnouine"/>
| <person person_id="2" fname="Sajon" lname="Rondela"/>
| <person person_id="3"><fname>Likame</fname><lname>Örrtmons</lname></person>
| <per |
+-----+-----+-----+-----+
|
|
|
+-----+-----+-----+-----+
1 row in set (0.04 sec)

mysql>
```

Result

Thus the XML Database is created and Validated

EX.NO:11 CREATING DOCUMENT,COLUMNS & GRAPH USING NOSQL

DATE :

Aim

To Create Document,column and Graph using NOSQL Tools.

Algorithm

Step 1:Start

Step 2:Create Database in MongoDB

Step 3:Create Collection and Document in MongoDB

Step 4:Display all document

Step 5:Stop

Create database in mongodb

>Install Mongodb shell

>Connect with localhost

>Connection string:

`mongodb://localhost:27017`

output:

```
mongosh mongodb://localhost:27017/?directConnection=true&serverSelectionTimeoutMS=2000
Please enter a MongoDB connection string (Default: mongodb://localhost/): mongodb://localhost:27017
mongodb://localhost:27017
Current Mongosh Log ID: 63f77936478602709ffec4c6
Connecting to:      mongodb://localhost:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+1
.7.1
Using MongoDB:      5.0.9
Using Mongosh:      1.7.1

For mongosh info see: https://docs.mongodb.com/mongodb-shell/

-----
  The server generated these startup warnings when booting
  2023-02-23T19:51:09.789+05:30: Access control is not enabled for the database. Read and write access to data and conf
  igation is unrestricted
  -----

  Enable MongoDB's free cloud-based monitoring service, which will then receive and display
  metrics about your deployment (disk utilization, CPU, operation statistics, etc).

  The monitoring data will be available on a MongoDB website with a unique URL accessible to you
  and anyone you share the URL with. MongoDB may use this information to make product
  improvements and to suggest MongoDB products and deployment options to you.

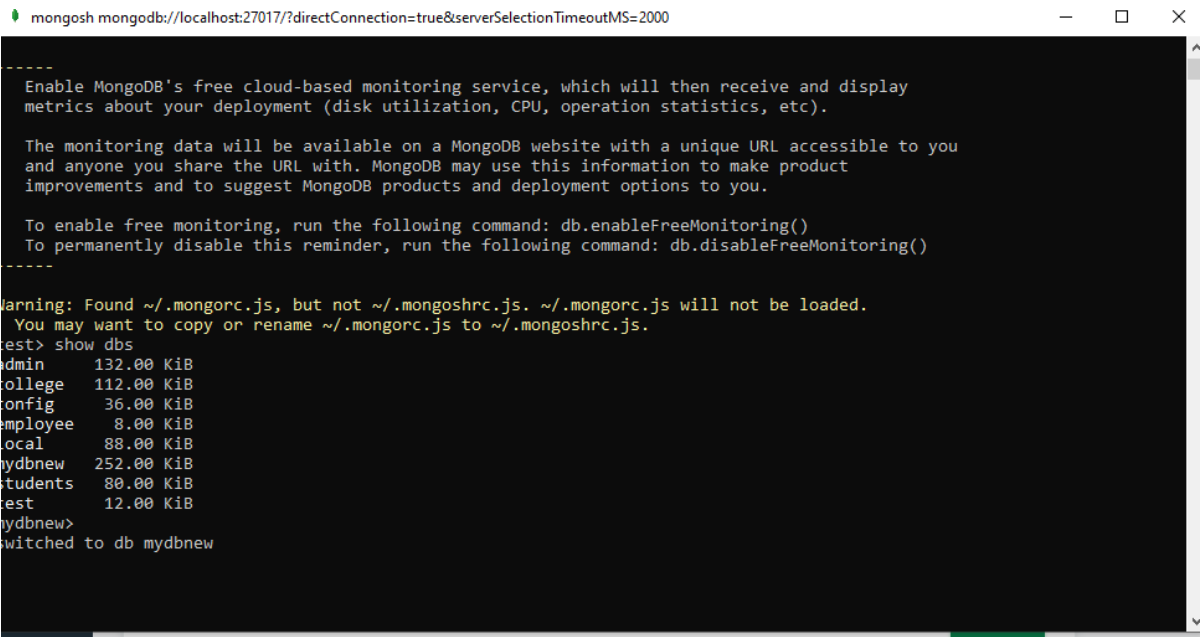
  To enable free monitoring, run the following command: db.enableFreeMonitoring()
  To permanently disable this reminder, run the following command: db.disableFreeMonitoring()
  -----

Warning: Found ~/.mongorc.js, but not ~/.mongoshrc.js. ~/.mongorc.js will not be loaded.
You may want to copy or rename ~/.mongorc.js to ~/.mongoshrc.js.
```

Create collection in mongodb

use <database_name> command

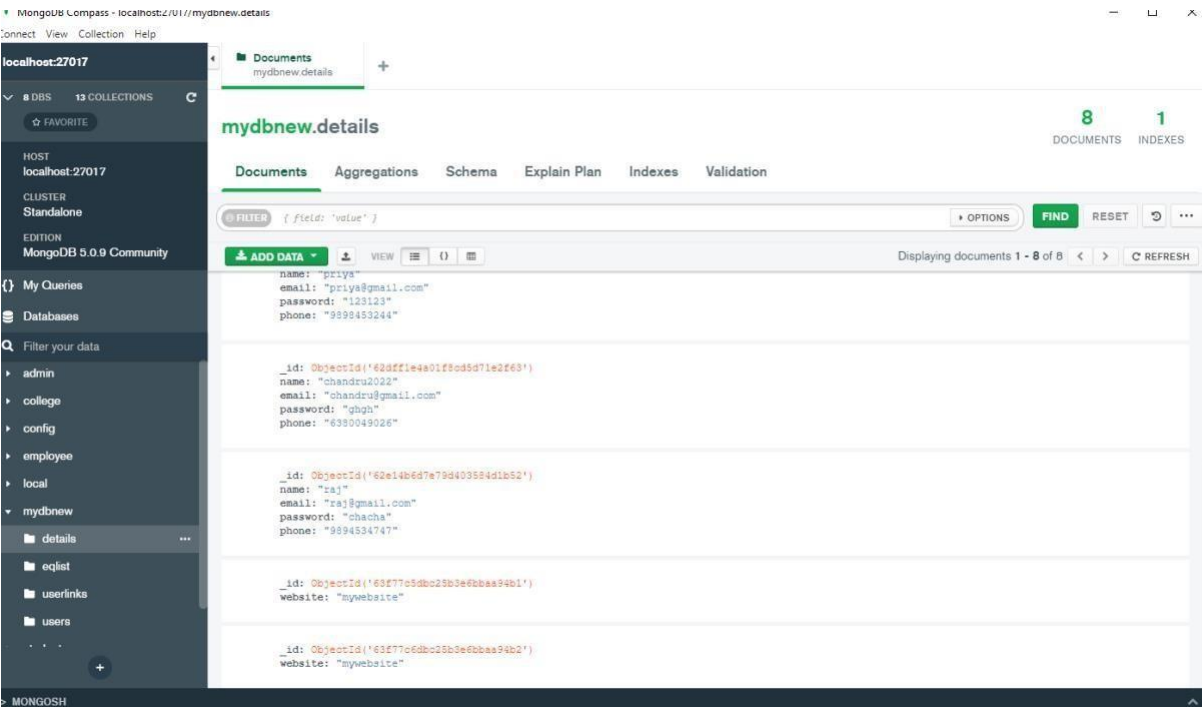
OUTPUT:



Create document in mongodb

mydbnew>db.details.insertOne({"website":"mywebsite"})

Output:



Display all documents

Db.details.find()

Output

```
mongosh mongodb://localhost:27017/?directConnection=true&serverSelectionTimeoutMS=2000
2 |
mydbnew> db.details.find();
[
  {
    _id: ObjectId("62c1bb5ff978763fbdac325b"),
    name: 'chandru',
    email: 'chandru009@gmail.com',
    password: 'chacha',
    phone: '9894532863'
  },
  {
    _id: ObjectId("62c1bbe515d7a438066b540c"),
    name: 'moorthy',
    email: 'moorthy@gmail.com',
    password: 'chacha123',
    phone: '6380049026'
  },
  {
    _id: ObjectId("62c1bf214d03b1bb8ab627bf"),
    name: 'kamatchi',
    email: 'kamatchi@gmail.com',
    password: 'demo',
    phone: '9894532863'
  },
  {
    _id: ObjectId("62c2d692f498c5d763ddf008"),
    name: 'priya',
    email: 'priya@gmail.com',
    password: '123123',
    phone: '9898453244'
  },
  {
    _id: ObjectId("62dffa1e4a01f8cd5d71e2f63"),
    name: 'chandru2022',
    email: 'chandru@gmail.com',
    password: 'ghgh',
    phone: '6380049026'
  },
  {
    _id: ObjectId("62e14b6d7e79d403584d1b52"),
    name: 'raj',
    email: 'raj@gmail.com',
    password: 'chacha',
    phone: '9894534747'
  },
  {
    _id: ObjectId("63f77c5dbc25b3e6bbaa94b1"),
    website: 'mywebsite'
  },
  {
    _id: ObjectId("63f77c6dbc25b3e6bbaa94b2"),
    website: 'mywebsite'
  }
]
```

CREATING CHART USING SAMPLE DATA

PROCEDURE:

Step 1: Log into MongoDB Atlas.

To access the MongoDB Charts application, you must be logged into Atlas

Step 2: Select your desired Atlas project, or create a new project.

If you have an Atlas Project with clusters containing data you wish to visualize,

Step 3: Select the project from the Context dropdown in the left navigation pane.

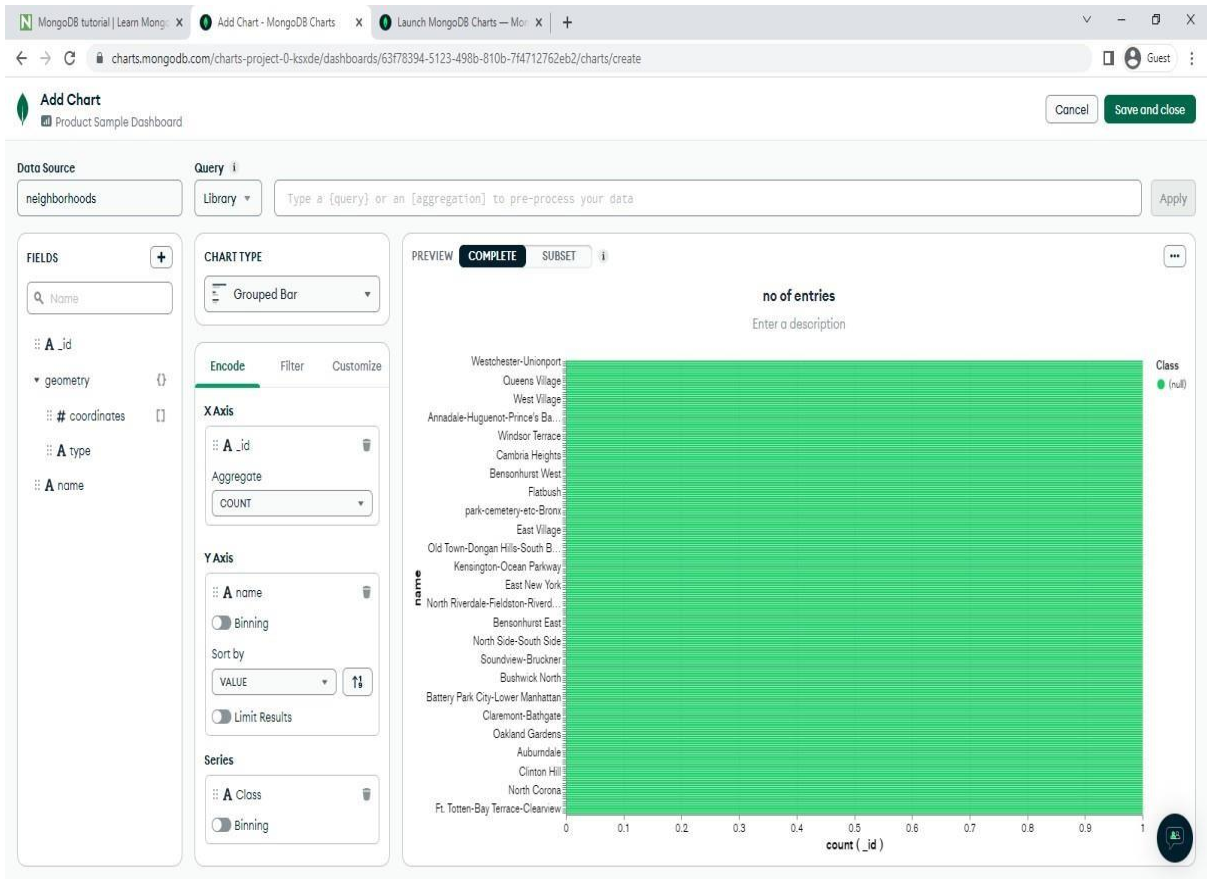
Step 4: Create an Atlas cluster. The MongoDB Charts application makes it easy to connect

Collections in your cluster as data sources. Data sources reference specific collections and charts views that you can access in the Chart Builder to visualize the data in those collections or charts views.

Step 5: Launch the MongoDB Charts application. In Atlas, click Charts in the navigation bar.

Step 6: Choose data from clusters

OUTPUT



Result

Thus the Document and Graph is created.

